Print preview in SAP UI5: generating a form on the fly via a RAP action

Fair warning: this guide will probably save a lot of developers a lot of time. It will even speed things up if you hand the task to an AI assistant, which would otherwise spend an hour confidently inventing APIs that never existed and then apologizing for it.

Print preview in SAP UI5 applications was a common requirement in our project implementation. There are many articles on this topic, but they are scattered, and reading them one by one never gives you the full picture. In this article I walk through an approach that I implemented in my last project: a PDF form is generated on request from the current state of an object and shown to the user in a print preview.

The article has two parts. The backend part (RAP) builds an action that returns the PDF content as a string. The frontend part (an application based on the List Report template) implements a custom action that invokes the RAP action, decodes the response and shows the PDF in a viewer.

Part 1. Backend

Example: printing a delivery note for an inbound delivery

Let's take the standard Inbound Delivery object and build a projection on top of it. The user selects one or more deliveries and gets a single PDF with delivery notes. Printing is needed both from the list and from the delivery's Object Page.

The backend solution has three parts: a projection view on `R_InboundDeliveryTP`, an abstract entity for the result, and a behavior definition with a single action. The form is built by the class `zcl_delnote_print` (it returns the generated PDF), and the PDFs of different deliveries are merged by `cl_rspo_pdf_merge`.

Step 1 (optional). Creating the business object

You can create a projection view or a new RAP business object (BO), or simply omit this step:

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Inbound delivery'

define root view entity ZC_TMP_InboundDelivery
  provider contract transactional_query
  as projection on R_InboundDeliveryTP
{
  key InboundDelivery,
      SDDocumentCategory,
      DeliveryDocumentType,
      ...
      OverallGoodsMovementStatus,
      ...
      TotalCreditCheckStatus
}
```

Step 2. Abstract entity for the result

An action has to return a file, so it needs a structured result. For that we create an abstract entity:

```
@EndUserText.label: 'Delivery note print form'

define abstract entity ZD_TMP_NOTE_PRINT
{
  FileName : bds_filena;
  MimeType : bds_mimetp;
  MimeCont : abap.string( 0 );
}
```

It has three fields: file name, MIME type and content. The content is declared as `abap.string( 0 )`, a string of unlimited length, and we put the PDF into it already encoded in base64 (see the implementation in Step 4).

Step 3. The action in the behavior definition

In the behavior definition of the business object, we declare an action that returns our abstract entity:

```
projection implementation in class zbp_c_tmp_inbounddelivery unique;

strict ( 2 );

define behavior for ZC_TMP_InboundDelivery
{
  use update;

  action ( authorization : global, features : instance ) printNote result [1]
ZD_TMP_NOTE_PRINT;
}
```

Three points to note:

- `features : instance` obliges you to implement the method `get_instance_features`. Without it the button stays disabled and the reason is not obvious. We use it to allow printing only for a certain status (Step 5).
- `authorization : global` requires the method `get_global_authorizations`. It can be left empty, but then the action is available to everyone who can open the application.
- `result [1]` means that the action returns a single result record.

Step 4. Implementing the action

The logic is as follows: read the selected deliveries, build a delivery note for each one, merge the PDFs into one file, encode it in base64 and return it in the result.

```
METHOD printnote.
  CHECK keys IS NOT INITIAL.

  READ ENTITIES OF zc_tmp_inbounddelivery IN LOCAL MODE
    ENTITY zc_tmp_inbounddelivery ALL FIELDS
    WITH CORRESPONDING #( keys )
    RESULT DATA(deliveries).

  DATA(pdf_merger) = NEW cl_rspo_pdf_merge( ).
```

```
  DATA(documents_added) = 0.

  LOOP AT deliveries ASSIGNING FIELD-SYMBOL(<delivery>).
    TRY.
        DATA(note) = NEW zcl_delnote_print(
          iv_delivery = <delivery>-%tky-inbounddelivery ).

        pdf_merger->add_document(
          document = note->get_pdf( )-pdf_content ).
        documents_added += 1.

      CATCH cx_somu_error INTO DATA(form_error).
        APPEND VALUE #( %tky = <delivery>-%tky )
          TO failed-zc_tmp_inbounddelivery.
        APPEND VALUE #( %tky = <delivery>-%tky
                        %msg = new_message_with_text(
                                 severity = if_abap_behv_message=>severity-error
                                 text     = form_error->get_text( ) ) )
          TO reported-zc_tmp_inbounddelivery.
    ENDTRY.
  ENDLOOP.

  IF documents_added = 0.
    RETURN.
  ENDIF.

  pdf_merger->merge_documents( IMPORTING merged_document = DATA(merged) ).

  DATA(pdf_merge_name) =
    |Note_{ cl_abap_context_info=>get_system_date( ) }{
cl_abap_context_info=>get_system_time( ) }.pdf|.
  DATA(pdf_merge_file) = cl_http_utility=>encode_x_base64( unencoded = merged ).

  INSERT VALUE #(
    %tky            = keys[ 1 ]-%tky
    %param-mimetype = 'application/pdf'
    %param-filename = pdf_merge_name
    %param-mimecont = pdf_merge_file )
  INTO TABLE result.
ENDMETHOD.
```

Things to pay attention to:

- **One file for several deliveries.** The PDFs of all selected deliveries are merged, so the result contains one record, bound to the first key (`keys[ 1 ]`).
- **Form errors.** If a delivery note could not be built for some delivery, the delivery goes into `failed` and the user gets a message through `reported`. The remaining deliveries are still printed. An empty `CATCH` would hide the problem: the file would simply lack the expected delivery note.

Step 5. When the button is available: instance features and authorizations

Printing a delivery note only makes sense for deliveries whose goods movement is complete. We set this condition in `get_instance_features`:

```
METHOD get_instance_features.
  READ ENTITIES OF zc_tmp_inbounddelivery IN LOCAL MODE
    ENTITY zc_tmp_inbounddelivery ALL FIELDS
    WITH CORRESPONDING #( keys )
    RESULT DATA(deliveries).

  LOOP AT deliveries ASSIGNING FIELD-SYMBOL(<delivery>).
    INSERT VALUE #(
      %tky              = <delivery>-%tky
      %action-printnote = COND #(
        WHEN <delivery>-overallgoodsmovementstatus EQ 'C'
        THEN if_abap_behv=>fc-f-unrestricted
        ELSE if_abap_behv=>fc-f-read_only ) )
      INTO TABLE result.
  ENDLOOP.
ENDMETHOD.

METHOD get_global_authorizations.
ENDMETHOD.
```

The status `C` of `OverallGoodsMovementStatus` means that the goods movement for the delivery is complete. For all other deliveries the action gets the `read_only` flag.

This result reaches the client through the service metadata. In `$metadata`, the action gets a `Core.OperationAvailable` annotation with the path `_it/__OperationControl/printNote`, and each delivery's data carries the Boolean property `__OperationControl/printNote`. On the frontend, this is what decides whether the button is enabled, and the frontend part below covers it.

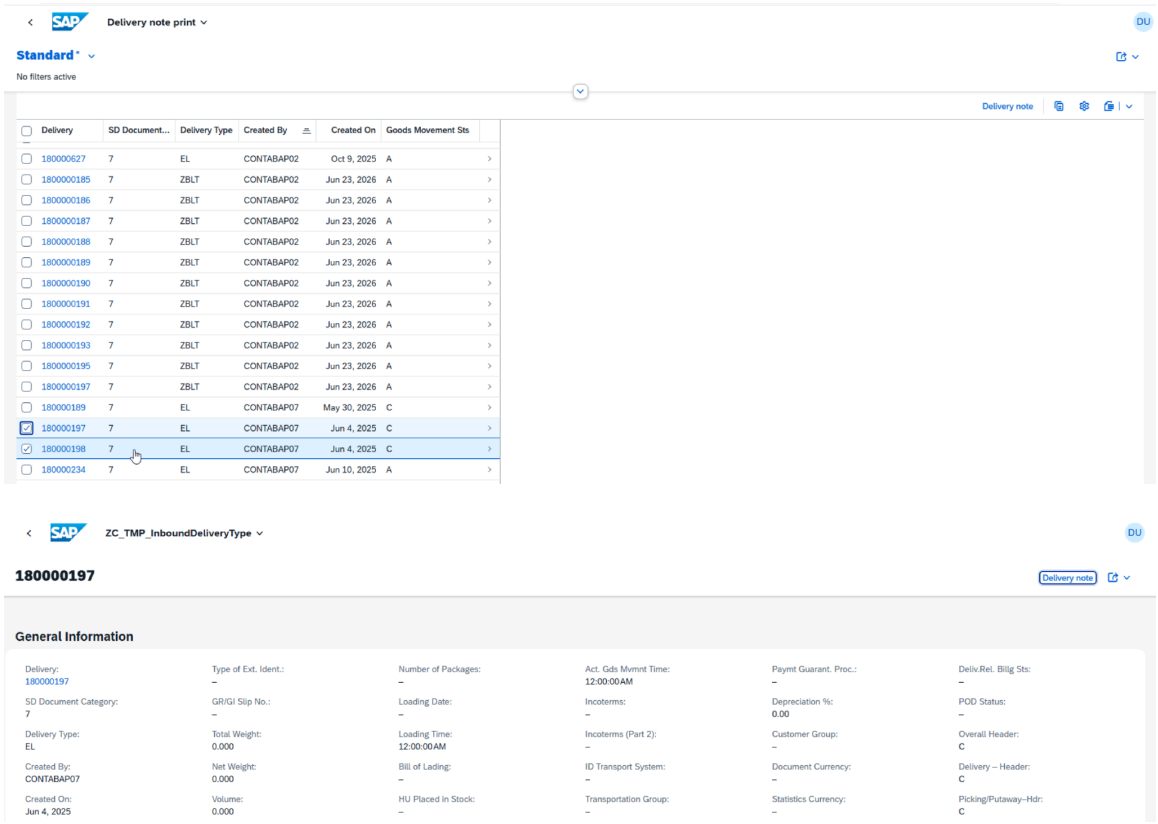
An important caveat: `__OperationControl/printNote` is a property from the metadata of our particular OData service, and in our service its name matches the name of the action. For your own actions you have to find this property yourself by looking at the `$metadata` of your service. I did not look into how exactly it is formed, and this article does not cover it.

A short note on selecting several rows in the list. In our example the button is enabled only if printing is allowed for all selected deliveries (on the frontend this is an `every` check in Step 7). If you would rather print everything the user is authorized to and silently skip the rest, use `some`. You will then have to duplicate the check in the action itself and skip the unsuitable deliveries on the backend. This is a separate topic and this article does not cover it.

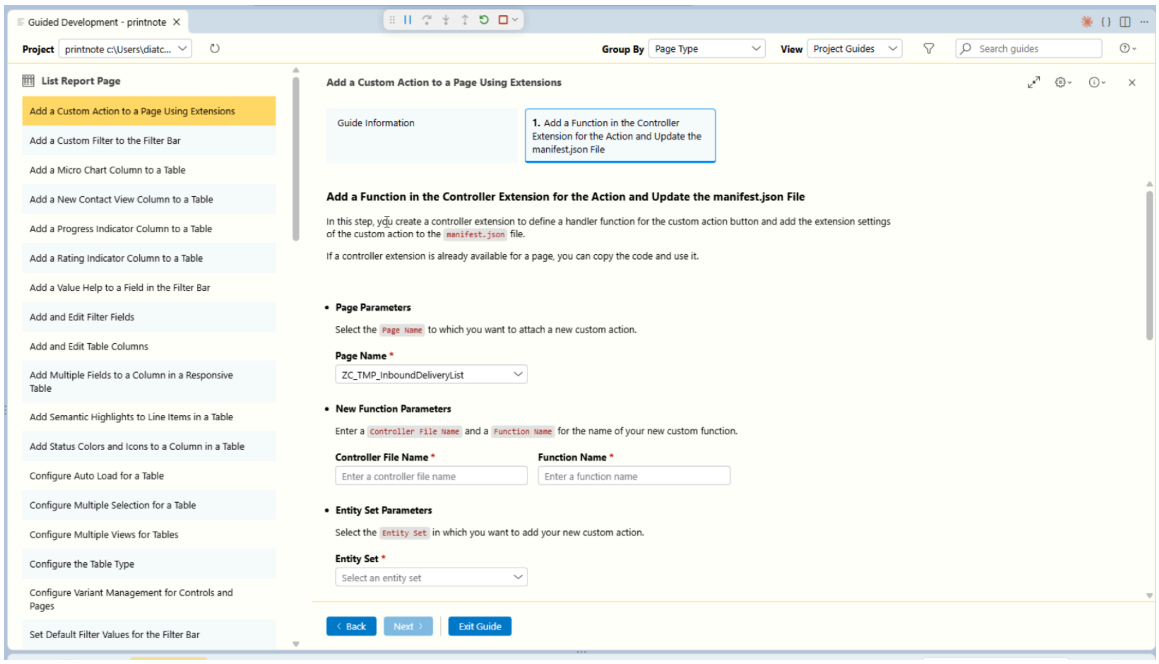
`get_global_authorizations` is empty in our example. If printing should not be available to everyone, the authorization check has to be implemented in this method.

Part 2. Frontend

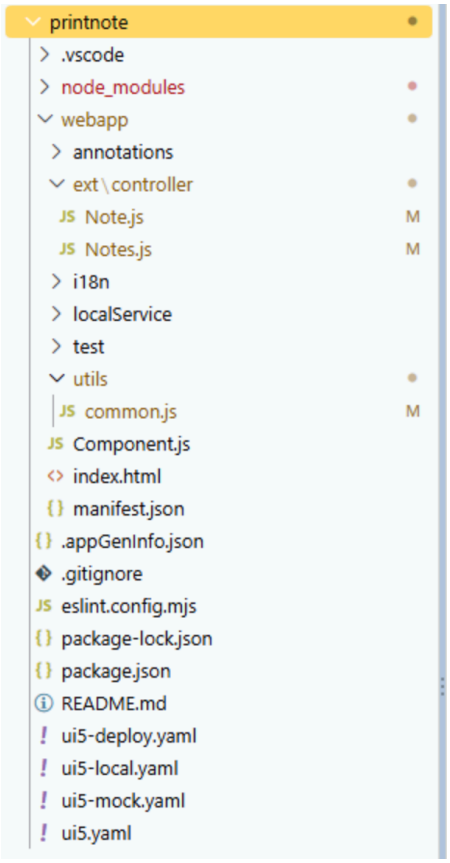
The application is based on the standard List Report template, with an OData V4 service as its data source and custom actions implemented as controller extensions in plain JavaScript.



On the frontend we need two buttons: one on the List Report page and one on the Object Page. You can use Guided Development to add custom actions to the List Report and Object Page.



After extending the controller and moving some common code into the `utils` folder, you should end up with the following project structure:



The buttons call custom actions that invoke the RAP BO action, decode the response and show the PDF using `sap.m.PDFViewer`.

Step 6. Declare the custom actions in the manifest

There are two entry points, and each one is a custom action in `manifest.json`.

In the List Report, the action is added to the table toolbar (path: `routing → targets → ZC_TMP_InboundDeliveryList → options → settings → controlConfiguration → @com.sap.vocabularies.UI.v1.LineItem`):

```
"actions": {
  "Notes": {
    "press": "your.domain.namespace.ext.controller.Notes.Print",
    "visible": true,
    "enabled": "your.domain.namespace.ext.controller.Notes.isEnabled",
    "requiresSelection": true,
    "text": "{@i18n>deliveryNote}"
  }
}
```

In the Object Page, the action goes into the header (path: `routing → targets → ZC_TMP_InboundDeliveryObjectPage → options → settings → content → header`):

```
"actions": {
  "Note": {
    "press": "your.domain.namespace.ext.controller.Note.Print",
```

```
    "visible": true,
    "enabled": "your.domain.namespace.ext.controller.Note.isEnabled",
    "requiresSelection": false,
    "text": "{@i18n>deliveryNote}"
  }
}
```

Step 7. Enable the button from the backend's feature control

In the backend part, the instance features produced the property `__OperationControl/printNote` for every delivery. Both buttons use it, through one shared helper (the full code is in Step 9, in the file `../utils/common.js`):

```
isEnabled: async function (oBindingContext, aSelectedContexts, sProperty) {
    const aContexts = [oBindingContext]
        .concat(aSelectedContexts || [])
        .filter(Boolean);

    if (!aContexts.length) {
        return false;
    }

    try {
        const aValues = await Promise.all(
            aContexts.map((oContext) => oContext.requestProperty(sProperty))
        );
        return aValues.every((bValue) => bValue === true);
    } catch (oError) {
        return false;
    }
}
```

In the Object Page there is one binding context, and in the List Report there are the selected rows. The helper requests the property for each context and enables the button only if it is `true` for all of them (`every`). As noted above, if you prefer to print everything the user is authorized to and skip the rest, use `some` and repeat the check in the action.

The same caveat as before applies: `__OperationControl/printNote` is a property of my service's metadata, so for your own action look up the name in your `$metadata`.

Step 8. Call the action

The Object Page handler is in the file `../ext/controller/Note.js`:

```
sap.ui.define([
    "your/domain/namespace/noteprint/utils/common"
], function (Common) {
    "use strict";

    return {
        Print: async function (oEvent) {
            var oContext = this.getBindingContext();
            var oModel = oEvent.getModel();

            var result = await
```

```
this.getEditFlow().invokeAction("com.sap.gateway.srvd.ztmp_ui_inbounddelivery.v0001.p
rintNote", {
                contexts: oContext,
                model: oModel
            });

            var oPDF = Array.isArray(result) ? result[0].value.getObject() :
result.getObject();

            Common.openPDFFromBase64(oPDF);
        },
        isEnabled: function (oBindingContext, aSelectedContexts) {
            return Common.isEnabled(oBindingContext, aSelectedContexts,
"__OperationControl/printNote");
        }
    };
});
```

The List Report handler is in the file `../ext/controller/Notes.js`:

```
sap.ui.define([
    "your/domain/namespace/noteprint/utils/common"
], function (Common) {
    "use strict";

    return {
        Print: async function ( ) {
            var oModel = this.getModel();
            var oExtensionAPI = this;
            var oTable =
oExtensionAPI.byId("your.domain.namespace.noteprint::ZC_TMP_InboundDeliveryList--fe::
table::ZC_TMP_InboundDelivery::LineItem::Table");
            var oItems = oTable.getSelectedContexts( );

            var result = await
this.getEditFlow().invokeAction("com.sap.gateway.srvd.ztmp_ui_inbounddelivery.v0001.p
rintNote", {
                contexts: oItems,
                model: oModel,
                invocationGrouping: "ChangeSet"
            });

            var oPDF = Array.isArray(result) ? result[0].value.getObject() :
result.getObject();

            Common.openPDFFromBase64(oPDF);
        },
        isEnabled: function (oBindingContext, aSelectedContexts) {
            return Common.isEnabled(oBindingContext, aSelectedContexts,
"__OperationControl/printNote");
        }
    };
});
```

Both handlers do the same thing: collect the contexts, call the action through the edit flow of the extension API, take the result object and pass it to the helper that displays the PDF.

- **The action name.** `invokeAction` needs the fully qualified name. In my service it is `com.sap.gateway.srvd.ztmp_ui_inbounddelivery.v0001.printNote`. Take yours from your `$metadata`.
- **Where the contexts come from.** In the Object Page it is the binding context of the page. In the List Report I read the selected rows from the table.
- **The table is found by its generated id.** The identifier `...::LineItem::Table` is built by Fiori elements and depends on the application id and the entity. In your application it will be different, so you have to find it yourself.
- **Invocation grouping.** In the List Report I pass `invocationGrouping: "ChangeSet"`. Unlike "Isolated", which sends the selected rows one after another, "ChangeSet" sends all the keys together in a single batch, so the backend can merge their PDFs in one call.

Step 9. Decode the response and show the PDF

The backend returns standard base64, so a plain `atob` is enough. The helper turns the string into a Blob, creates an object URL and opens it in a `PDFViewer`. The full file is `../utils/common.js`:

```
sap.ui.define([
    "sap/m/PDFViewer",
    "sap/m/MessageBox"
], function (PDFViewer, MessageBox) {
    "use strict";

    let oViewer = null;
    let sBlobUrl = null;

    function base64ToBlob(sBase64, sMimeType) {
        const aBytes = Uint8Array.from(atob(sBase64), (c) => c.charCodeAt(0));
        return new Blob([aBytes], { type: sMimeType || "application/pdf" });
    }

    return {
        openPDFFromBase64: function (oPDF) {
            if (!oPDF || !oPDF.MimeCont) {
                return null;
            }

            try {
                const oBlob = base64ToBlob(oPDF.MimeCont, oPDF.MimeType);

                if (sBlobUrl) {
                    URL.revokeObjectURL(sBlobUrl);
                }
                sBlobUrl = URL.createObjectURL(oBlob);

                if (!oViewer) {
                    oViewer = new PDFViewer({
                        error: function () {
                            MessageBox.error("The PDF could not be displayed.");
                        }
                    });
                }
```

```
                oViewer.setSource(sBlobUrl);
                oViewer.setTitle(oPDF.FileName || "Document");
                oViewer.open();

                return oViewer;
            } catch (oError) {
                MessageBox.error("The PDF could not be opened.");
                return null;
            }
        },

        isEnabled: async function (oBindingContext, aSelectedContexts, sProperty) {
            const aContexts = [oBindingContext]
                .concat(aSelectedContexts || [])
                .filter(Boolean);

            if (!aContexts.length) {
                return false;
            }

            try {
                const aValues = await Promise.all(
                    aContexts.map((oContext) => oContext.requestProperty(sProperty))
                );
                return aValues.every((bValue) => bValue === true);
            } catch (oError) {
                return false;
            }
        }
    };
});
```

Summary

To sum up, to build a print preview on the fly you need to do the following:

On the backend:

1. Create an abstract entity with a file name, a MIME type and the content.
2. Create a RAP BO action definition that returns this abstract entity.
3. Create the RAP BO action implementation that builds the PDF, encodes it in base64 and returns it.

On the frontend:

1. Extend the application with a custom action on either the List Report page or the Object Page.
2. Link the availability of the custom action to the feature control of the RAP BO action.
3. Create a handler that calls the RAP BO action through the edit flow and reads the result.
4. Create a helper that decodes the base64 string into a Blob and shows it in one reusable `PDFViewer`, without leaving objects behind.

An example can be found in https://github.com/diatchkov/printnote